# SINCRONIZAÇÃO

Conceitos básicos, Problemas e Resoluções

**By: Artemísia Kimberlly Silva**

# Relembrando...

**Programa:**

É apenas um arquivo estático salvo no disco rígido (SSD/HD). Ele é um conjunto de instruções escritas em código (como um arquivo .exe ou um binário compilado). Ele não está "vivo"; não consome memória RAM nem CPU.

**Processo:**

É o programa em execução. Quando você clica para abrir um programa, o Sistema Operacional lê o arquivo no disco, aloca uma área isolada na memória RAM (para guardar as variáveis e o código) e prepara o ambiente para ele rodar.

**Thread:**

É a unidade ativa de execução de CPU dentro de um processo. Um processo pode ter apenas uma thread (monothread) ou várias threads trabalhando juntas (multithread). O ponto crucial aqui é: todas as threads dentro de um mesmo processo compartilham a mesma memória RAM e os mesmos recursos desse processo.

## Sincronização entre Processos

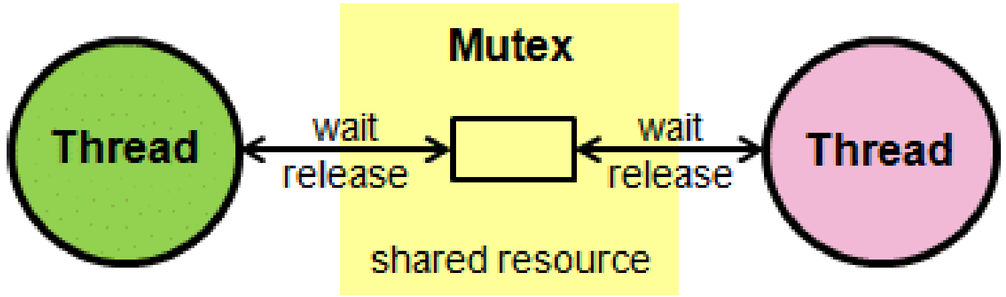
Em sistemas modernos, os processos não rodam isolados; eles frequentemente cooperam e compartilham recursos como memória e arquivos para realizar tarefas complexas. No entanto, sem uma coordenação rígida e precisa do tempo, o paralelismo físico ou lógico gera o caos nos dados

## Concorrência e Compartilhamento de Memória

Tarefas cooperantes precisam interagir. Threads pertencentes ao mesmo processo compartilham nativamente a mesma memória física, o que torna a troca de dados extremamente veloz. Para processos distintos, o sistema operacional cria canais de comunicação (IPC). Mas lembrem-se da distinção fundamental que discutimos: o fato de existir um canal ou memória compartilhada resolve a comunicação (transmissão de dados), mas não a coordenação (ordenação no tempo)."

## Recurso Compartilhado:

É qualquer entidade do sistema que pode ser acessada e manipulada por mais de um processo ou thread de forma concorrente. Isso pode ser um componente de hardware (como uma porta serial, um periférico ou memória compartilhada) ou uma estrutura lógica (como uma variável global, um arquivo, ou um banco de dados).

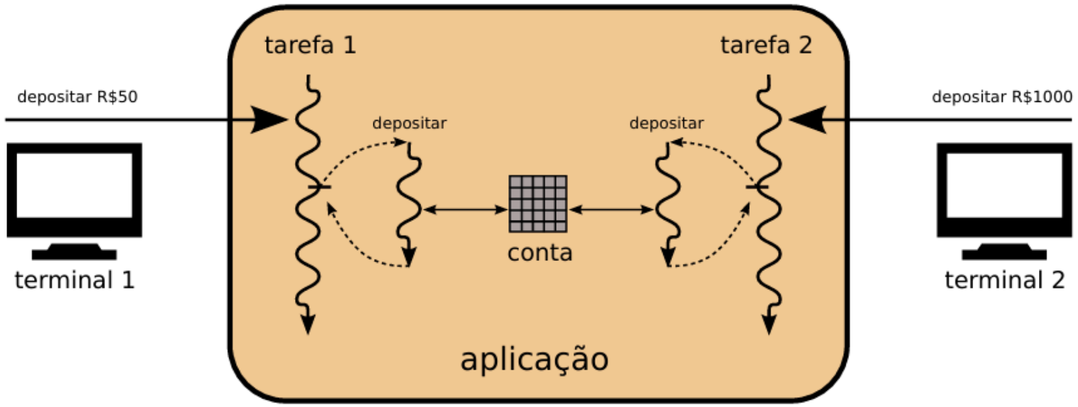





A caixa de giz de cera ou, mais especificamente, uma cor muito disputada, como o giz vermelho. Ambas as crianças precisam dessa mesma cor para completar seus desenhos.

## Condição de Corrida:

É a anomalia ou falha que ocorre exatamente quando a seção crítica não é protegida adequadamente. Acontece quando duas ou mais threads manipulam dados compartilhados de forma simultânea, e o resultado final passa a depender da ordem exata (e imprevisível) em que o escalonador do Sistema Operacional alterna a execução delas. As threads literalmente "apostam corrida".






Acontece se não houver a regra da professora e as duas crianças tentarem pegar o giz vermelho exatamente ao mesmo tempo. Elas se chocam, disputam o mesmo giz, puxam cada uma para um lado, e o giz quebra ao meio. O resultado é um desastre (corrupção de dados) gerado pela falta de coordenação no tempo.

## Seção Crítica:

Considere um sistema consistindo em n processos {P0 , P1 ,..., Pn-1 }. Cada processo possui um segmento de código, chamado seção crítica (critical section), onde o processo pode alterar variáveis comuns, atualizando uma tabela, gravando um arquivo, e assim por diante. O recurso importante do sistema é que, quando um processo está executando em sua seção crítica, nenhum outro processo pode ter permissão para executar em sua seção crítica, isto é, dois processos não podem executar em suas seções críticas ao mesmo tempo. O problema da seção crítica é projetar um protocolo que os processos possam utilizar para a cooperação. Cada processo precisa solicitar permissão para entrar em sua seção crítica. [Silberschatz, em Sistemas Operacionais em Java]

```
1  void depositar (long conta, long *saldo, long valor)
2  {
3     enter (conta) ;          // entra na seção crítica "conta"
4     (*saldo) += valor ;      // usa as variáveis compartilhadas
5     leave (conta) ;          // sai da seção crítica
6  }
```

USANDO AS PRIMITIVAS ENTER() E LEAVE(),

## Seção Crítica:

Para que qualquer solução de sincronização (como travas de software ou hardware) seja considerada correta e robusta, ela deve satisfazer obrigatoriamente três critérios fundamentais:

### 1. EXCLUSÃO MÚTUA

**É a regra de ouro da concorrência. Se um processo P1 está executando sua seção crítica, nenhum outro processo pode estar executando em suas respectivas seções críticas que envolvam o mesmo recurso compartilhado.**

### 2. PROGRESSO

**Se nenhum processo está executando em sua seção crítica e existem processos desejando entrar nela, a escolha de qual processo entrará a seguir não pode ser adiada indefinidamente.**

### 3. ESPERA LIMITADA

**Deve haver um limite para o número de vezes que outros processos têm permissão para furar a fila e entrar em suas seções críticas após um processo ter feito a solicitação para entrar na sua.**

## Exclusão Mútua:

É a garantia fundamental de que, se um processo está utilizando um recurso compartilhado, nenhum outro processo poderá acessá-lo ao mesmo tempo. O trecho de código onde o recurso compartilhado é manipulado é conhecido como seção crítica. Diversos mecanismos podem ser definidos para garantir a exclusão mútua, impedindo o entrelaçamento de seções críticas. Todos eles exigem que o programador defina os limites (início e o final) de cada seção crítica [MAZIERO, ].



Só uma pessoa pode segurar o giz vermelho por vez". Se a Criança A pegou o giz, a Criança B é obrigada a cruzar os braços e esperar até que a Criança A termine e devolva o giz para a caixa (liberando o acesso).

## Condições de Bernstein:

Condições de disputa entre tarefas paralelas podem ser formalizadas através das chamadas condições de Bernstein [Bernstein, 1966], assim definidas: dadas duas tarefas t1 e t2, sendo R(ti) o conjunto de variáveis lidas por ti e W(ti) o conjunto de variáveis escritas por ti, essas tarefas podem executar em paralelo sem risco de condição de disputa (t1 || t2) se e somente se as seguintes condições forem atendidas:

$$t_1 \parallel t_2 \iff \begin{cases} \mathcal{R}(t_1) \cap \mathcal{W}(t_2) = \varnothing & (t_1 \text{ não lê as variáveis escritas por } t_2) \\ \mathcal{R}(t_2) \cap \mathcal{W}(t_1) = \varnothing & (t_2 \text{ não lê as variáveis escritas por } t_1) \\ \mathcal{W}(t_1) \cap \mathcal{W}(t_2) = \varnothing & (t_1 \text{ e } t_2 \text{ não escrevem nas mesmas variáveis}) \end{cases}$$

**Exclusão mútua**: somente uma tarefa pode estar dentro da seção crítica em cada instante.

**Espera limitada:** uma tarefa que aguarda acesso a uma seção crítica deve ter esse acesso garantido em um tempo finito, ou seja, não pode haver inanição.

I**ndependência de outras tarefas:** a decisão sobre o uso de uma seção crítica deve depender somente das tarefas que estão tentando entrar na mesma. Outras tarefas, que no momento não estejam interessadas em entrar na região crítica, não podem influenciar sobre essa decisão.

I**ndependência de fatores físicos:** a solução deve ser puramente lógica e não depender da velocidade de execução das tarefas, de temporizações, do número de processadores no sistema ou de outros fatores físicos.

## O Limite das Soluções Simples (E por que elas falham)

As abordagens mais primárias para garantir a Exclusão Mútua tentam resolver o problema via software (como o Algoritmo de Peterson) ou com instruções especiais de hardware (como Test-and-Set ou Compare-and-Swap).

O princípio básico dessas soluções é usar uma variável de trava (um lock):

```
WHILE (LOCK == TRUE) {
   // FAÇA NADA, APENAS ESPERE
}
LOCK = TRUE;
// --- SEÇÃO CRÍTICA ---
LOCK = FALSE;
```

**POR QUE ISSO É INEFICIENTE? A FALHA CATASTRÓFICA DESSA ABORDAGEM ARQUITETURAL É JUSTAMENTE A ESPERA OCUPADA (BUSY WAITING OU SPINLOCK).**

**Algoritmo de Peterson (Exclusão Mútua via Software)**

***Conceito:*** Solução puramente por software para 2 processos (P0 e P1) que garante exclusão mútua sem hardware especial.

***Mecanismo:*** Usa um vetor flag[2] (intenção de entrar) e a variável turn (a preferência/cortesia).

***Código:***

```
flag[i] = true;          // Declara intenção
turn = j;                // Cede a preferência (cortesia)
while (flag[j] && turn == j) {
    // Espera ocupada (busy waiting)
}
flag[i] = false;         // Libera ao sair
```

## Instruções Atómicas de Hardware (TAS e CAS)

***O Problema da Abordagem Simples:*** No código assembly, testar e atribuir uma variável consome múltiplos ciclos de clock. Uma interrupção entre a leitura e a escrita quebra a exclusão mútua.

***Test-and-Set (TAS):***

Lê o valor antigo e define como true numa única operação indivisível (atómica).

```
// Primitiva de Spinlock com TAS:
while (TestAndSet(&lock)) {
    // Espera ocupada...
}
lock = false;
```

Quando um processo tenta entrar na seção crítica e ela está ocupada, ele entra em um loop infinito, testando a variável continuamente. Isso gera dois problemas graves:

- **Desperdício de CPU**: O processo gasta fatias valiosas de tempo do processador (ciclos de CPU) apenas para descobrir que ainda não pode avançar, roubando processamento que poderia ser usado pelo próprio processo que está dentro da seção crítica (o que atrasa a liberação do recurso).

- **Problema da Inversão de Prioridade:** Se o processo na seção crítica tem prioridade baixa e o processo em espera ocupada tem prioridade alta, o escalonador pode dar toda a CPU para o de alta prioridade, impedindo que o processo de baixa prioridade termine e libere a trava. O sistema entra em colapso.

Para resolver a "espera ocupada", ficou claro que a sincronização não poderia ser resolvida apenas pelo programa girando no vácuo; o Sistema Operacional precisava intervir. É aqui que Edsger Dijkstra introduziu os Semáforos (1965). Em vez de queimar CPU, um semáforo diz ao SO: "Se o recurso não estiver disponível, me tire da CPU e me acorde só quando ele for liberado."

Um semáforo S é uma variável inteira que, após a inicialização, só pode ser acessada por duas operações atômicas padrão (frequentemente chamadas de wait() e signal(), ou P e V):

- **wait(S):** Se o semáforo for > 0, ele subtrai 1 e o processo continua. Se for 0, o processo é bloqueado (colocado na fila de espera do SO, liberando a CPU).
- **signal(S)**: Incrementa o semáforo em 1. Se houver processos dormindo na fila deste semáforo, o SO acorda um deles.

## SEMÁFOROS

Além de eliminar a espera ocupada, semáforos podem ser inicializados com valores maiores que 1 (Semáforos Contadores). Isso os torna perfeitos não apenas para exclusão mútua de um recurso, mas para gerenciar um pool de recursos (como um buffer de 10 posições em um problema de Produtor-Consumidor).

Mas qual o problema?

Se um programador cometer um erro simples ao escrever o código:

- Esquecer um signal() > Todos os processos que derem wait() dormirão para sempre **(Deadlock).**
- Inverter a ordem de signal() e wait() > Vários processos entram na seção crítica ao mesmo tempo **(Condição de Corrida)**.

Como a variável compartilhada e o semáforo que a protege não têm nenhuma ligação formal no código (são variáveis separadas), o compilador não consegue ajudar a evitar esses erros.

## MONITORES

Para proteger o sistema dos erros dos programadores, pesquisadores como Tony Hoare e Per Brinch Hansen desenvolveram os Monitores. A premissa dos Monitores é elevar a sincronização para o nível da linguagem de programação, encapsulando-a. Um monitor é um Tipo Abstrato de Dados (como uma Classe em orientação a objetos) que agrupa:

1. Os dados compartilhados (estado).
2. Os métodos (funções) que operam sobre esses dados.
3. A lógica de sincronização.

A linguagem de programação garante automaticamente que apenas uma thread pode estar ativa dentro do monitor a qualquer momento. O programador não precisa escrever wait() ou signal() para garantir a exclusão mútua; basta chamar o método do monitor, e a própria estrutura garante que não haverá concorrência interna.

# MUTEXES

Um mutex (abreviação de Mutual Exclusion, ou Exclusão Mútua) é uma primitiva de sincronização essencial no design de sistemas operacionais e programação concorrente. Ele atua como um mecanismo de bloqueio ("lock") para garantir que apenas uma thread ou processo por vez tenha acesso a um recurso compartilhado, protegendo assim a chamada região crítica.

Quando múltiplas threads tentam ler e modificar variáveis globais, estruturas de dados ou dispositivos de hardware simultaneamente, podem ocorrer condições de corrida (race conditions), resultando em dados corrompidos ou comportamentos imprevisíveis. O mutex previne isso serializando o acesso.

# MUTEXES

O ciclo de vida básico de um mutex envolve duas operações atômicas principais (geralmente apoiadas por instruções de hardware, como Test-and-Set ou Compare-and-Swap):

1. Lock (Bloqueio): A thread solicita o acesso. Se o mutex estiver livre (destravado), a thread o adquire e continua a execução. Se já estiver em posse de outra thread, a thread solicitante é bloqueada (colocada em estado de sleep ou espera) pelo escalonador do sistema operacional até que o mutex seja liberado.
2. Unlock (Liberação): Após concluir a operação na região crítica, a thread libera o mutex, permitindo que uma das threads que estava na fila de espera seja acordada e adquira o lock.

Problemas clássicos de coordenação

Retratam situações de coordenação típicas em muitos sistemas. Descrevem soluções genéricas e eficientes de problemas usuais.

Exemplos:

- Produtores/consumidores
- Leitores/escritores
- O barbeiro dorminhoco
- Jantar dos filósofos

# TRABALHO DA SEGUNDA UNIDADE

## *1. Objetivo do Projeto*

Este trabalho prático tem como objetivo consolidar os conceitos de Seção Crítica, Condição de Corrida, Deadlock e Inanição (Starvation). Vocês deverão implementar algoritmos clássicos de sincronização de processos e threads, contrastando as vantagens e limitações de diferentes mecanismos estruturais: Semáforos, Mutexes e Monitoes.

## *2. Distribuição dos Temas*

A turma (24 alunos) será dividida em 8 trios. Cada trio ficará responsável por um problema clássico e um mecanismo de sincronização específico.

## TRABALHO DA SEGUNDA UNIDADE

***3. Requisitos Técnicos:*** O projeto exige simulação de tempo real, uso de logs descritivos no terminal e linguagens específicas para cada mecanismo (C/C++ para semáforos/mutexes e Java/C#/Python para monitores).

***4. Entregáveis***: Repositório no GitHub com código comentado e README, vídeo de demonstração de 3 a 5 minutos (incluindo o "teste do caos" ao desativar a sincronização) e apresentação em sala de aula.

***5. Prazos:*** Envio do repositório e do vídeo até 8 de outubro, com apresentações marcadas para 9 de outubro.